IRREDUCIBLE MODULI: A FORMALLY VERIFIED RESIDUE NUMBER SYSTEM OVER ARBITRARY MODULI

ARTUR FLAMANDZKI

Abstract. We define a residue number system over arbitrary moduli, composite and not pairwise coprime moduli included. Residues are computed by counting, greatest common divisors by Euclid's algorithm on residues, and the moduli are combined into a mixed-radix system in which each modulus m receives the radix $m/\gcd(W, m)$, where W is the product of the radices before it. No definition refers to primes, to factorization, or to a number assembled from residues. We prove that the product of the radices is the least common multiple of the moduli, in any order; that on the consecutive moduli $2, \ldots, n-1$ the modulus n receives the radix p when $n = p^k$ and 1 otherwise, so it receives its whole value exactly when n is prime, and the logarithm of the radix is the von Mangoldt function. Numbers given only by their residues can be compared, tested for primality, added, subtracted, multiplied, raised to powers and divided exactly, with overflow and underflow detected; and the residue computed by counting is also read from a comb of hyperbolic-secant pulses from scale 3 on. Counting multiplicatively modulo a prime q by a primitive root takes equal values at the same pairs of steps as counting modulo $q-1$, and a nonzero residue is a square exactly when its step is even. The work states several independent topics in one formal frame: remainders, Euclid's algorithm, the least common multiple of consecutive integers and the von Mangoldt function, primes in intervals of consecutive integers, and residue number system arithmetic over arbitrary moduli. Every statement in this paper is a theorem of the accompanying Lean 4 / Mathlib formalization, which builds with no `sorry` and uses only the axioms `propext`, `Quot.sound` and `Classical.choice`.

1. Introduction

A residue number system represents a number by its residues modulo a list of moduli. This paper defines such a system over arbitrary moduli, using counting modulo a modulus as the only primitive, and proves what it computes.

The definitions (Section 2) use only moduli and residues below a modulus. Primes, factorizations and numbers assembled from residues do not occur in them. They occur only in the theorems (Section 3), which identify the computed quantities with the usual arithmetic ones: residues computed by counting with remainders, Euclid's algorithm on residues with greatest common divisors, the product of the radices with the least common multiple of the moduli, and the moduli that receive their whole value as radix after the consecutive moduli below them with the primes.

Purpose and contributions. The aim is one frame in which several topics that are usually treated separately are stated with the same few notions and verified by the same proof checker. Within that frame, the work contributes three points that the author did not find in the literature:

(C1) A machine-checked residue number system with its non-modular operations: magnitude comparison, detection of overflow and underflow, exact division, and a primality test, all on the residues of a number (T5, T5c, T6, T8). The author found formal proofs of the Chinese remainder theorem, but no formal verification of comparison or mixed-radix conversion in a residue number system.

(C2) Arbitrary moduli. The moduli need not be pairwise coprime: each modulus m receives the radix $m/\gcd(W, m)$, only the part of m that the moduli before it do not already cover (T3), and comparison works on these radices (T5). The comparison methods the author found assume pairwise coprime moduli.

(C3) No number is assembled from residues. Every computation goes through residues; the only number formed from residues is the residue of a block modulus, which is below that modulus (Section 2.4).

In the same frame the work proves several known results (T1, T2, T4, T7, T9) from the definitions rather than restating them, and adds the reading of residues from a sech comb (T1w).

The comparison with earlier work reflects the literature known to the author at the time of writing; it states what the author did not find, not that no such work exists.

Related work. Theorems T2–T4 are classical. The identity $\operatorname{lcm}(1,\dots,n) = e^{\psi(n)}$, with the jump by p exactly at the powers of p, is formalized in Isabelle by Eberl [2] and is part of Mathlib [4] as `Chebyshev.psi_eq_log_lcmUpto`; Mathlib also contains the Chinese remainder theorem for moduli that are not coprime (`Nat.chineseRemainder'`). Here these facts are derived from definitions that do not mention them.

Residue number systems go back to Garner [3]. Addition and multiplication act on each residue separately, while comparison, overflow detection and division need a positional representation of the number, usually through mixed-radix conversion; a recent method and an account of the earlier ones are given in [1]. Moduli that are not coprime are used in error-correcting residue codes [6]. T9 restates in terms of counting two classical facts: for a primitive root g modulo a prime q, $t \mapsto g^t \bmod q$ maps $\mathbb{Z}/(q-1)$ onto the nonzero residues, and by Euler's criterion (in Mathlib as `ZMod.euler_criterion`) a nonzero residue g^t is a square exactly when t is even. For the reading of residues from a sech comb (T1w) the author found no earlier work.

Every theorem below is stated with the name of the Lean theorem that proves it. Hypotheses are given in full; nothing here is weaker or stronger than the Lean statement. The formalization and the build instructions are described in Section 4.

Notation. All numbers are natural numbers. We write $x \bmod 0 = x$ (the Lean convention), and $\lfloor x/y \rfloor$ for integer division, with $\lfloor x/0 \rfloor = 0$. A list of moduli is written $R = (r_1, \dots, r_k)$; "positive moduli" means $r_i \geq 1$ for every i. We write $W(R) = \operatorname{lcm}(r_1, \dots, r_k)$, with $W = 1$ for the empty list; the residue vector of x on R is $[x]_R = (x \bmod r_1, \dots, x \bmod r_k)$; and $\prod R = r_1 \cdots r_k$. The list R followed by $m_1, \dots, m_j$ is written $(R, m_1, \dots, m_j)$. Λ is the von Mangoldt function.

2. Definitions

2.1. Residues by counting and Euclid's algorithm.

Definition 1. The residue of t modulo d is computed by counting alone: add 1 per step and reset to 0 on reaching d,

$$\operatorname{countMod}(d, 0) = 0, \qquad \operatorname{countMod}(d, t+1) = \begin{cases} 0 & \text{if } \operatorname{countMod}(d,t) + 1 = d, \\ \operatorname{countMod}(d,t) + 1 & \text{otherwise,} \end{cases}$$

and $\operatorname{resid}(a, b) = \operatorname{countMod}(b, a)$.

Lean: `countMod`, `resid`

Definition 2. Euclid's algorithm on residues:

$$\operatorname{euclid}(a, 0) = a, \qquad \operatorname{euclid}(a, b) = \operatorname{euclid}\big(b, \operatorname{resid}(a, b)\big) \quad (b \geq 1).$$

In Lean the recursion carries a fuel bound of $b + 1$ steps, which is always enough.

Lean: `euclid`

2.2. **The mixed-radix system.**

Definition 3. The residue modulo c of the product of a list of radices is computed factor by factor, without forming the product:

$$\mathrm{prodMod}((), c) = 1 \bmod c, \qquad \mathrm{prodMod}((r, R'), c) = \big(\mathrm{resid}(r, c) \cdot \mathrm{prodMod}(R', c)\big) \bmod c.$$

The gcd of m with that product is

$$\mathrm{gcdProd}(R, m) = \mathrm{euclid}\big(\mathrm{prodMod}(R, m), m\big).$$

Lean: `prodMod`, `gcdProd`

Definition 4. The mixed-radix system of the moduli $m_1, \ldots, m_k$ in this order has the radices $L_k = \mathrm{radices}(m_1, \ldots, m_k)$, where $L_0 = ()$ and

$$L_{j+1} = \Big(L_j,\ \frac{m_{j+1}}{\mathrm{gcdProd}(L_j, m_{j+1})}\Big).$$

The radix modulus m receives after the moduli B is

$$\mathrm{newRadix}(B, m) = \frac{m}{\mathrm{gcdProd}(\mathrm{radices}(B), m)},$$

and m is irreducible after B, $\mathrm{IrreducibleAfter}(B, m)$, when $\mathrm{newRadix}(B, m) = m$: it receives its whole value as radix. Finally $\mathrm{initialModuli}(n) = (2, 3, \ldots, n-1)$.

Lean: `radices`, `newRadix`, `IrreducibleAfter`, `initialModuli`

Example 1. Checked by the Lean kernel (`decide`):

$$\mathrm{radices}(2,3,4,5,6,7,8,9,10) = (2,3,2,5,1,7,2,3,1),$$
$$\mathrm{radices}(6,35) = (6,35), \qquad \mathrm{radices}(4,4) = (4,1).$$

2.3. **Mixed-radix representations and comparison.** A mixed-radix representation is a list of pairs of a digit and a radix, least significant first: $D = \big((d_1, r_1), \ldots, (d_k, r_k)\big)$.

Definition 5. The residue modulo c of the value of D is computed by Horner's rule from the most significant digit, with every radix reduced modulo c:

$$\mathrm{evalMod}((), c) = 0,$$
$$\mathrm{evalMod}\big(((d, r), D'), c\big) = \big(d + \mathrm{resid}(r, c) \cdot \mathrm{evalMod}(D', c)\big) \bmod c.$$

The value of D is $\mathrm{eval}(D) = \sum_{i=1}^{k} d_i \prod_{j<i} r_j$. It is formed only as the residue of a block modulus (Section 2.4), where it is below that modulus.

Lean: `evalMod`, `eval`

Definition 6. $\mathrm{invMod}(a, b)$ is the least $x \geq 1$ with $xa \equiv 1 \pmod b$, found by repeated addition of a modulo b (at most b additions; $\mathrm{invMod}(a, 1) = 0$). $\mathrm{toMixedRadix}(R, S)$ converts the residues S on the moduli R to a mixed-radix representation, modulus by modulus (Garner's step on arbitrary moduli): modulus m with residue s, appended to the representation D built so far, with $g = \mathrm{gcdProd}(\text{radices of } D, m)$ and $r = m/g$, takes $y = (s - \mathrm{evalMod}(D, m)) \bmod m$; if $g \nmid y$ the residues are incompatible and the result is none; otherwise the new radix is r and the new digit is $(y/g) \cdot \mathrm{invMod}\big((\mathrm{prodMod}(\text{radices of } D, m)/g) \bmod r,\ r\big) \bmod r$ (digit 0 when $r = 1$). $\mathrm{compareRes}(R, S, S')$ converts both residue vectors and compares the two representations digit by digit from the most significant.

Lean: `invMod`, `toMixedRadix`, `compareRes`

2.4. **Blocks.** A block is a list of moduli used as one modulus: its modulus is the lcm of its members, and its residue is the value of the mixed-radix representation of the member residues.

Definition 7. A block of members M with residues S is the modulus and residue

$$\text{blockModulus}(M,S) = (\textstyle\prod \text{radices of } D,\ \text{eval}(D)), \qquad D = \text{toMixedRadix}(M,S),$$

and blockModuli applies blockModulus to each block of a list. Base extension, $\text{baseExtend}(R,S,T)$, takes $D = \text{toMixedRadix}(R,S)$ and gives $\text{evalMod}(D,c)$ for each modulus c in T.

Lean: `blockModulus`, `blockModuli`, `baseExtend`

Definition 8. A number in block representation (BlockRep) is given by the member residues of its blocks and, for its quotient by the lcm of those blocks, the member residues of its quotient blocks. flatten turns it into one list of moduli: the block moduli followed by the quotient block moduli, where a quotient block of modulus c with residue k receives the residue $(k \cdot \text{prodMod}(\text{radices of the blocks}, c) + \text{evalMod}(\text{representation of the blocks}, c)) \bmod c$. $\text{compareBlocks}(x,y)$ flattens both numbers, extends both to the moduli of x followed by the moduli of y not among them, and compares them there with compareRes.

For statements only, $\text{BlockRep.of}(n,D,K)$ writes n in blocks D and quotient blocks K: the members m of D receive the residues $n \bmod m$, those of K the residues $\lfloor n/L \rfloor \bmod m$, where L is the lcm of all members of D.

Lean: `BlockRep`, `flatten`, `compareBlocks`, `BlockRep.of`

2.5. **Primality on residues.**

Definition 9. A number is given by its residues S on moduli R. Trial divisors $m = 2, 3, \ldots,$ at most fuel of them, are compared with it through their own residue vectors $(\text{resid}(m,c))_{c\in R}$ and $(\text{resid}(m,c)^2 \bmod c)_{c\in R}$, while the residue of the number modulo m is computed with evalMod from its mixed-radix representation. $\text{isPrimeRes}(\text{fuel}, R, S)$ is true when m equals the number or $m \cdot m$ exceeds it, false when the number is below m or divisible by m, and none when the fuel runs out or the residues are incompatible.

Lean: `isPrimeRes`

2.6. **Atoms of an interval.**

Definition 10. The differences of the interval $lo, \ldots, hi$ are

$$\text{Diff}(lo,hi) = \{\, k \ge 2 : \exists a :\ lo \le a,\ k \mid a,\ a + k \le hi \,\}.$$

Lean: `IntervalDifference`

Definition 11. $\text{firstMultipleFrom}(lo,k)$ is the least multiple of k that is at least lo, computed from one residue: lo if $\text{resid}(lo,k) = 0$, and $lo + k - \text{resid}(lo,k)$ otherwise. The atoms of the interval $lo, \ldots, hi$, $\text{intervalAtoms}(lo,hi)$, are the differences $k = 2, \ldots, hi - lo$, taken in order, that are kept when the interval contains a and $a+k$ with $a = \text{firstMultipleFrom}(lo,k)$, and k is divisible by none of the atoms kept before. The definition does not refer to divisibility order; Theorem T7 shows that the atoms are the minimal elements of $\text{Diff}(lo,hi)$ under divisibility, that is, the atoms of $\text{Diff}(lo,hi)$ ordered by divisibility.

Lean: `firstMultipleFrom`, `intervalAtoms`

2.7. **Arithmetic on residues.**

Definition 12. For each modulus m with residues a and b: addRes gives $(a+b) \bmod m$, subRes gives $(a+(m-b)) \bmod m$, mulRes gives $ab \bmod m$; powRes performs e products starting from the residues of 1. A sum overflows, addOverflow, when its residue vector compares below that of the first term; a difference underflows, subUnderflow, when the first term compares below the second.

Lean: `addRes`, `subRes`, `mulRes`, `powRes`, `addOverflow`, `subUnderflow`

Definition 13. For a divisor t each modulus m is replaced by $m/\mathrm{euclid}(m,t)$; we write $R/t = \mathrm{divModuli}(R,t)$ for the resulting list. On modulus m with $g = \mathrm{euclid}(m,t)$ and $r = m/g$, exactDivRes maps the residue s to $(s/g) \cdot \mathrm{invMod}\big((t/g) \bmod r,\ r\big) \bmod r$. $\mathrm{divModRes}(R,S,T)$ computes the remainder ρ modulo T with evalMod, subtracts ρ modulus by modulus and divides exactly by T. addressRes gives the address (i,k) with the offsets counted $1,\dots,T$: remainder 0 becomes offset T, and otherwise the quotient is increased by one.

Lean: `divModuli`, `exactDivRes`, `divModRes`, `addressRes`

2.8. **The sech comb.**

Definition 14. Modulus d is represented by a sum of sech pulses of width $1/s$ centred at the multiples of d, where s is the scale:

$$\mathrm{sech}(x) = \frac{1}{\cosh(x)}, \qquad C_{s,d}(t) = \sum_{i=-1}^{2} \mathrm{sech}\Big(s\big(t - d(\lfloor t/d \rfloor + i)\big)\Big).$$

A step t is a peak for d when $C_{s,d}(t) > \frac{1}{2}$. $\mathrm{lastPeak}_s(d,t)$ is the last peak for d among the steps $1,\dots,t$ (0 if there is none), and $\mathrm{combResid}_s(a,b) = a - \mathrm{lastPeak}_s(b,a)$ reads the residue of a modulo b backwards from a to the last peak for b.

Lean: `sech`, `sechComb`, `lastPeak`, `combResid`

Definition 15. For real x, y,

$$\mathrm{sechRatio}(x,y) = \frac{2xy}{x^2+y^2}.$$

Lean: `sechRatio`

2.9. **Multiplicative counting.**

Definition 16. The residue of g^t modulo q is computed by multiplicative counting: start at 1, multiply by g per step and take the residue modulo q,

$$\mathrm{mulCount}(g,q,0) = \mathrm{resid}(1,q), \qquad \mathrm{mulCount}(g,q,t+1) = \mathrm{resid}\big(\mathrm{mulCount}(g,q,t) \cdot g,\ q\big).$$

Lean: `mulCount`

3. Results

T1 Counting.

Theorem 1. *For every d and t, $d = 0$ included,* $\mathrm{countMod}(d,t) = t \bmod d$. *Hence* $\mathrm{resid}(a,b) = a \bmod b$.

Lean: `countMod_eq_mod`, `resid_eq_mod`

T1w Sech comb.

Theorem 2. *For scale $s \geq 3$, every modulus $d \geq 1$ and every step t,*

$$C_{s,d}(t) > \tfrac{1}{2} \iff d \mid t.$$

Hence for $s \geq 3$ and $b \geq 1$, $\mathrm{combResid}_s(a,b) = \mathrm{resid}(a,b)$. At $s = 2$ the reading fails: $\mathrm{combResid}_2(3,2) = 0 \neq 1 = \mathrm{resid}(3,2)$.

Lean: `one_half_lt_sechComb_iff`, `combResid_eq`, `combResid_two_ne`

Theorem 3. *For all real a, b, $\mathrm{sech}(a-b) = \mathrm{sechRatio}(e^a, e^b)$; hence, with $X = e^{st}$ and $Y = e^{su}$,*

$$\mathrm{sech}(s(t-u)) = \frac{2XY}{X^2+Y^2}.$$

For $x, y > 0$, $\mathrm{sechRatio}(x,y) \leq 1$, with equality exactly when $x = y$; in particular $2x^n y/(x^{2n}+y^2) = 1$ exactly when $y = x^n$.

Lean: `sech_sub_eq_sechRatio`, `sech_mul_sub_eq_sechRatio`, `sechRatio_le_one`, `sechRatio_eq_one_iff`, `sechRatio_pow_eq_one_iff`

Theorem 4. *For $x > 0$ the comb at scale $\ln x$ is a sum of ratios of integer powers of x,*

$$C_{\ln x,d}(t) = \sum_{i=-1}^{2} \mathrm{sechRatio}\big(x^t,\ x^{d(\lfloor t/d \rfloor + i)}\big),$$

and $3 \leq \ln x$ exactly when $e^3 \leq x$. Hence for $x \geq e^3$, every modulus $d \geq 1$ and every step t, $C_{\ln x,d}(t) > \frac{1}{2} \iff d \mid t$.

Lean: `sechComb_log_eq`, `three_le_log_iff`, `one_half_lt_sechComb_log_iff`

T2 Euclid.

Theorem 5. $\mathrm{euclid}(a,b) = \gcd(a,b)$.

Lean: `euclid_eq_gcd`

T3 Mixed-radix system.

Theorem 6. *For any list R and any c, m:*

$$\mathrm{prodMod}(R,c) = \textstyle\prod R \bmod c, \qquad \mathrm{gcdProd}(R,m) = \gcd\big(\textstyle\prod R, m\big),$$

$$\mathrm{radices}(R,m) = \Big(\mathrm{radices}(R),\ \frac{m}{\gcd\big(\prod \mathrm{radices}(R), m\big)}\Big).$$

Lean: `prodMod_eq`, `gcdProd_eq`, `radices_append`

Theorem 7. *For positive moduli R and a list B of positive moduli,*

$$\textstyle\prod \mathrm{radices}(R) = W(R), \qquad \mathrm{newRadix}(B,m) = \dfrac{m}{\gcd(W(B),m)},$$

and for $m \geq 1$, $\mathrm{IrreducibleAfter}(B,m) \iff \gcd(W(B),m) = 1$.

Lean: `prod_radices`, `newRadix_eq`, `irreducibleAfter_iff`

So the product of the radices of positive moduli is their least common multiple, in any order of the moduli, and a modulus receives its whole value as radix exactly when it is coprime to the lcm of the moduli before it.

T4 Consecutive moduli.

Theorem 8. *For $n \geq 2$,*

$$\mathrm{newRadix}(\mathrm{initialModuli}(n), n) = \begin{cases} p & \text{if } n = p^k,\ p \text{ prime},\ k \geq 1, \\ 1 & \text{otherwise,} \end{cases}$$

$$\mathrm{IrreducibleAfter}(\mathrm{initialModuli}(n), n) \iff n \text{ is prime},$$
$$\log \mathrm{newRadix}(\mathrm{initialModuli}(n), n) = \Lambda(n).$$

Lean: `newRadix_initialModuli`, `irreducibleAfter_initialModuli_iff`, `log_newRadix_initialModuli`

T5 Comparison.

Theorem 9. *For every mixed-radix representation D and modulus c,* $\mathrm{evalMod}(D, c) = \mathrm{eval}(D) \bmod c$. *For $b > 1$ and $\gcd(a, b) = 1$, $a \cdot \mathrm{invMod}(a, b) \equiv 1 \pmod{b}$.*

Lean: `eval_append`, `evalMod_eq`, `invMod_spec`

Theorem 10. *For positive moduli R and every n there is a mixed-radix representation $D = ((d_i, r_i))_i$ with*

$$\mathrm{toMixedRadix}(R, [n]_R) = D, \qquad (r_i)_i = \mathrm{radices}(R), \qquad d_i < r_i, \qquad \mathrm{eval}(D) = n \bmod W(R).$$

For positive moduli R and $n, n' < W(R)$, $\mathrm{compareRes}(R, [n]_R, [n']_R) = \mathrm{compare}(n, n')$.

Lean: `toMixedRadix_spec`, `compareRes_eq`

T5c Blocks.

Theorem 11. *For positive moduli M,* $\mathrm{blockModulus}(M, [n]_M) = \big(W(M),\ n \bmod W(M)\big)$*; for blocks $P_1, \ldots, P_k$ of positive moduli,* blockModuli *gives the moduli $W(P_1), \ldots, W(P_k)$ with the residues $n \bmod W(P_i)$. For positive moduli R, any moduli T and $n < W(R)$,* $\mathrm{baseExtend}(R, [n]_R, T) = [n]_T$.

Lean: `blockModulus_spec`, `blockModuli_spec`, `baseExtend_spec`

Theorem 12. *For blocks $D_1, \ldots, D_a$ and quotient blocks $K_1, \ldots, K_b$ of positive moduli, with $Q = (W(D_1), \ldots, W(D_a), W(K_1), \ldots, W(K_b))$,*

$$\mathrm{flatten}(\mathrm{BlockRep.of}(n, D, K)) = (Q, [n]_Q).$$

If moreover n is below the lcm of all moduli of D, K and n' below that of all moduli of D', K' (blocks of positive moduli), then

$$\mathrm{compareBlocks}\big(\mathrm{BlockRep.of}(n, D, K),\ \mathrm{BlockRep.of}(n', D', K')\big) = \mathrm{compare}(n, n').$$

Lean: `flatten_spec`, `compareBlocks_eq`

T6 Primality.

Theorem 13. *Let R be positive moduli with $n < W(R)$ and $2 < W(R)$. Every answer b of* $\mathrm{isPrimeRes}(\mathrm{fuel}, R, [n]_R)$ *is true exactly when n is prime, and an answer is given once*

$$\mathrm{fuel} > n \quad \text{or} \quad \Big(\mathrm{fuel} > \lfloor \sqrt{n} \rfloor \text{ and } \big(\lfloor \sqrt{n} \rfloor + 1\big)^2 < W(R)\Big).$$

Lean: `isPrimeRes_sound`, `isPrimeRes_eq`

T7 Interval atoms.

Theorem 14. *For $k \geq 1$,* $\mathrm{firstMultipleFrom}(lo, k)$ *is the least multiple of k that is $\geq lo$, and it lies in $[lo, lo + k)$. For all lo, hi, k,*

$$k \in \mathrm{intervalAtoms}(lo, hi) \iff k \text{ is prime} \ \wedge\ \exists a :\ lo \leq a,\ k \mid a,\ a + k \leq hi.$$

So every atom is prime (`prime_of_mem_intervalAtoms`), and a prime p with $lo + 2p \leq hi + 1$ is an atom: an interval of L consecutive integers yields every prime p with $2p \leq L$.

Lean: `firstMultipleFrom_spec`, `mem_intervalAtoms`, `prime_mem_intervalAtoms`

Theorem 15. *Every divisor $d \geq 2$ of an element of* $\mathrm{Diff}(lo, hi)$ *is in* $\mathrm{Diff}(lo, hi)$*, and the atoms are the minimal elements of* $\mathrm{Diff}(lo, hi)$ *under divisibility:*

$$k \in \mathrm{intervalAtoms}(lo, hi) \iff k \in \mathrm{Diff}(lo, hi) \ \wedge\ \forall d \in \mathrm{Diff}(lo, hi) :\ d \mid k \implies d = k.$$

Lean: `intervalDifference_of_dvd`, `mem_intervalAtoms_iff_minimal`

Example 2. Checked by the Lean kernel: $\mathrm{intervalAtoms}(100, 130) = (2, 3, 5, 7, 11, 13, 17)$ and $\mathrm{intervalAtoms}(2, 60) = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29)$.

T8 Arithmetic.

Theorem 16. *For any moduli R and all x, y, e:*

$$\mathrm{addRes}(R, [x]_R, [y]_R) = [x + y]_R, \quad \mathrm{mulRes}(R, [x]_R, [y]_R) = [xy]_R, \quad \mathrm{powRes}(R, [x]_R, e) = [x^e]_R.$$

Lean: `addRes_spec`, `mulRes_spec`, `powRes_spec`

In the rest of this subsection R is a list of positive moduli and $W = W(R)$.

Theorem 17. *If $y \leq x$ then* $\mathrm{subRes}(R, [x]_R, [y]_R) = [x - y]_R$*; if $y \leq W$ then* $\mathrm{subRes}(R, [x]_R, [y]_R) = [x + (W - y)]_R$*. For $x, y < W$,*

$$\mathrm{addOverflow}(R, [x]_R, [y]_R) \iff W \leq x + y, \qquad \mathrm{subUnderflow}(R, [x]_R, [y]_R) \iff x < y.$$

Lean: `subRes_spec_of_le`, `subRes_spec`, `addOverflow_iff`, `subUnderflow_iff`

Theorem 18. *For $t \geq 1$ with $t \mid x$,* $\mathrm{exactDivRes}(R, [x]_R, t) = [x/t]_{R/t}$*, where $R/t = (r_i / \gcd(r_i, t))_i$; and if $x < W$ then $x/t < W(R/t)$.*

Lean: `exactDivRes_spec`, `div_lt_divModuli`

Theorem 19. *For $T \geq 1$ and $x < W$,* $\mathrm{divModRes}(R, [x]_R, T) = (x \bmod T,\ [\lfloor x/T \rfloor]_{R/T})$*. For $0 < x < W$, with $i = (x - 1) \bmod T + 1$ and $k = \lfloor (x - 1)/T \rfloor + 1$, so that $x = T(k - 1) + i$ and $1 \leq i \leq T$,*

$$\mathrm{addressRes}(R, [x]_R, T) = (i,\ [k]_{R/T}).$$

Lean: `divModRes_spec`, `addressRes_spec`

Theorem 20. *For any moduli R, a prime p with $p^v \mid W(R)$ and $p^{v+1} \nmid W(R)$, and every j,*

$$W(R, p^{v+1}, p^{v+2}, \ldots, p^{v+j}) = p^j\, W(R).$$

Lean: `lcm_append_prime_powers`

T9 Multiplicative counting.

Theorem 21. *For all g, q, t,* $\mathrm{mulCount}(g, q, t) = g^t \bmod q$.

Lean: `mulCount_eq`

In the rest of this subsection q is prime and g is a primitive root modulo q, that is, the order of g in $\mathbb{Z}/q$ is $q - 1$.

Theorem 22. *The multiplicative counter modulo q and the additive counter modulo $q - 1$ take equal values at the same pairs of steps:*

$$\mathrm{mulCount}(g, q, t) = \mathrm{mulCount}(g, q, u) \iff \mathrm{countMod}(q - 1, t) = \mathrm{countMod}(q - 1, u),$$

and every residue $0 < r < q$ equals $\mathrm{mulCount}(g, q, t)$ *for some $t < q - 1$. The counter returns to* 1 *exactly at the multiples of $q - 1$:* $\mathrm{mulCount}(g, q, t) = 1 \iff (q - 1) \mid t$, *which for scale $s \geq 3$ is $C_{s,q-1}(t) > \frac{1}{2}$.*

Lean: `mulCount_eq_mulCount_iff`, `exists_mulCount_eq`, `mulCount_eq_one_iff`, `one_half_lt_sechComb_iff_mulCount_eq_one`

Theorem 23. *Let $q \neq 2$, $q \nmid N$, and* $\mathrm{mulCount}(g, q, t) = N \bmod q$. *Then N is a square modulo q exactly when t is even.*

Lean: `isSquare_iff_even`

4. The formalization

The formalization is a Lean 4 [5] project built against Mathlib [4], both at version `v4.35.0`. All definitions of Section 2 except the sech comb are in `lean/Irreducibility/Defs.lean`, which does not import Mathlib; the sech comb needs real numbers and is defined in `SechComb.lean` and `SechRatio.lean`. Each group of results has its own file:

T1 Counting	`Residues.lean`	T5 Comparison	`Compare.lean`
T1w Sech comb	`SechComb.lean`	T5c Blocks	`Blocks.lean`
T1w ratio form	`SechRatio.lean`	T6 Primality	`Primality.lean`
T2 Euclid	`Euclid.lean`	T7 Interval atoms	`Atoms.lean`
T3 Mixed-radix system	`Radices.lean`	T8 Arithmetic	`Arithmetic.lean`
T4 Consecutive moduli	`InitialModuli.lean`	T9 Multiplicative	`Multiplicative.lean`

`lake build` completes with no `sorry`, `admit` or `native_decide`, and the theorems depend only on the axioms `propext`, `Quot.sound` and `Classical.choice`. The examples quoted above are checked by the kernel with `decide`. To build:

```
cd lean
lake exe cache get
lake build
```

5. Use of AI

The construction (residues computed by counting, the mixed-radix system of arbitrary moduli built modulus by modulus, block representations, the residue read from a sech comb, the irreducibility of a modulus after other moduli, the atoms of an interval, arithmetic on residues) and the choice of the statements to be proved are the author's. Generative AI (Claude Opus 5.5, Anthropic, used through Claude Code) contributed in the following ways:

- It wrote the Lean definitions from the author's description of the construction, and the Lean proofs. The author checked the definitions against the construction; the correctness of the proofs rests on the Lean kernel.

- It wrote the statements in standard notation and drafted this text. The author set the structure of the paper and the stated contributions.
- It searched the literature and checked the identifiers of the references. The author decided how the work is placed relative to that literature.

References

[1] L.-S. Didier, L. Glandus, N. El Mrabet and J.-M. Robert, *Residue number system comparison revisited, a software perspective*, 2025 5th International Conference on Electrical, Computer and Energy Technologies (ICECET), 2025; arXiv:2605.18415.

[2] M. Eberl, *Elementary facts about the distribution of primes*, Archive of Formal Proofs, February 2019, ISSN 2150-914x, `https://isa-afp.org/entries/Prime_Distribution_Elementary.html`.

[3] H. L. Garner, *The residue number system*, IRE Transactions on Electronic Computers EC-8 (1959), no. 2, 140–147, doi:10.1109/TEC.1959.5219515.

[4] The mathlib Community, *The Lean mathematical library*, Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), 367–381, doi:10.1145/3372885.3373824.

[5] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, Automated Deduction – CADE 28, Lecture Notes in Computer Science, 2021, 625–635, doi:10.1007/978-3-030-79876-5_37.

[6] S. Sundaram and C. N. Hadjicostis, *Fault-tolerant convolution via Chinese remainder codes constructed from non-coprime moduli*, IEEE Transactions on Signal Processing 56 (2008), no. 9, 4244–4254, doi:10.1109/TSP.2008.924636.